



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



1. Sintaxis de JavaScript

JavaScript es un lenguaje de programación creado por Netscape con el objetivo de integrarse en HTML y facilitar la creación de páginas interactivas sin necesidad de utilizar scripts de CGI (método por el cual un servidor web puede interactuar con programas externos de generación de contenido) o Java.

No hay que confundir Java con JavaScript. Java es un lenguaje completo que permite crear aplicaciones independientes y necesita ser compilado, mientras que JavaScript es un lenguaje que funciona como extensión del HTML. Es un lenguaje orientado a objetos, diseñado para el desarrollo de aplicaciones cliente – servidor a través de Internet.

El lenguaje JavaScript se basa en el estándar ECMAScript (que a partir del año 2015 a la actualidad se encarga de **regir como debe ser interpretado y funcionar el lenguaje JavaScript**, siendo este (JS – JavaScript) interpretado y procesado por multitud de plataformas, entre las que se encuentran los navegadores web y otros ambientes como el desarrollo de aplicaciones para los distintos sistemas operativos que actualmente existen en el mercado) y cada año hay actualizaciones por lo que recibe mejoras constantemente.

El código de programa de JavaScript, llamado **script**, se introduce directamente en el documento HTML y no necesita ser compilado, es el propio navegador el que se encarga de traducir dicho código.

JavaScript se ejecuta directamente en el navegador (cliente) de manera que éste pueda efectuar determinadas operaciones o tomar decisiones sin necesidad de acceder al servidor.

Por ejemplo, el desarrollo de un programa que verifique la clave de acceso a “x” sistema, JavaScript debe comprobar la información proporcionada por el usuario, verificar que sea correcta y actuar en consecuencia.

Aunque en la actualidad casi todos los navegadores admiten JavaScript, hay que tener en cuenta que los usuarios pueden inhabilitar el soporte JavaScript de su navegador.

Ahora bien, ya sabemos que un programa en JavaScript no debe ser compilado, ya que el navegador se encarga de leerlo cuando se carga la página, pero esto no siempre es así, puesto que en algunas ocasiones las funciones se ejecutan hasta que el usuario realiza una acción determinada, como, por ejemplo, pulsar sobre un link.

Una vez definido el doctype del documento, deberemos diferenciar dentro del mismo lo que es JavaScript del resto. Para ello HTML dispone de las etiquetas de inicio y final de código JavaScript:

```
<script language="javascript" type="text/javascript">  
</script>
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



El atributo **language** indica al navegador el lenguaje script utilizado, en nuestro caso debe ser JavaScript. Si éste se omite, el navegador asumirá que el script está programado en JavaScript.

Para que quede más claro, veremos nuestro primer programa con JavaScript:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>.: Javascript :.</title>

    <script lang="javascript" type="text/javascript">
      document.write("Mi primer programa con Javascript");
    </script>
  </head>

  <body>
  </body>
</html>
```

Éste muestra en pantalla una línea con el texto: Mi primer programa con JavaScript.

El código JavaScript se puede colocar en la cabecera o en el cuerpo del documento. Poco a poco, iremos encontrando las ubicaciones idóneas para ello.

Ten muy presente que JavaScript es un lenguaje de programación **case sensitive**, por lo tanto, una variable llamada Resultado es completamente distinta a otra llamada resultado.

Además, debemos considerar que no todos los navegadores soportan JavaScript por lo que las etiquetas **<script>** y **</script>** son ignoradas y se presenta el código script en pantalla al entender que se trata de un texto. La solución es añadir dos comentarios al inicio y final del script o bien, utilizar las etiquetas **<noscript>** y **</noscript>**, entre estas últimas se debe especificar una alternativa para los navegadores que no soportan scripts, como se muestra en la siguiente imagen:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>.: Javascript :.</title>
    <script lang="javascript" type="text/javascript">
      document.write("Mi primer programa con Javascript");
      //Para agregar un comentario
      /* Si tu comentario
      es de varias líneas de código
      */
    </script>
    <noscript>
      Tu navegador no soporta Javascript
    </noscript>
  </head>

  <body>
  </body>
</html>
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Aprovecho el código anterior para mostrarles que hay dos formas de incluir los comentarios:

- ✓ Los comentarios en una sola línea irán precedidos de `//`;
- ✓ Los comentarios de varias líneas irán encerrados en `/* y */`.

Las **llaves** se utilizan para definir fragmentos de código de manera que éstos no se junten con el resto del programa. Por ejemplo, todo el código que ejecuta una función.

```
6      <script lang="JavaScript" type="text/JavaScript">
7          for(cuenta=1; cuenta<4; cuenta++){
8              alert("Ventana de prueba\n Me veo:" + cuenta);
9          }
10     </script>
```

El **punto y coma** en JavaScript es separar las sentencias que se encuentran en una misma línea. También puede indicar el final de una sentencia que ocupa varias líneas.

Como hemos visto hasta ahora, introducimos el código JavaScript dentro del documento HTML; sin embargo, podemos crear un script externo mediante el atributo **src** aplicado a la etiqueta **script**, por ejemplo:

```
<script type= "text/javascript" src= "fecha.js"> .... </script>
```

Utilizar archivos externos es la mejor opción para trabajar con HTML siempre que sea posible, ya que nos permitirá reutilizar el código en otros documentos.

El script debe estar en un archivo de texto sin las etiquetas **<script>** y **</script>** y deberá tener la extensión **.js**.

A continuación, veremos un script que obtiene la fecha local y el cual fue vinculado en el ejemplo anterior:

```
1  //Mi primer script externo para calcular la fecha actual
2
3  var hoy = new Date();
4  dia = hoy.getDate();
5  mes = hoy.getMonth()+1;
6  anio = hoy.getFullYear();
7  alert ("Hoy es " + dia + " del mes " + mes + " de " + anio);
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Variables

Como en todo lenguaje de programación, las variables son fundamentales. JavaScript, admite cualquier tipo de nombre para definir una variable, solo deberás considerar lo siguiente:

- ✓ Primer carácter debe ser siempre una **letra**, el **guión bajo** o el símbolo **\$**., los caracteres restantes pueden ser letras, números o el guion bajo, cuidando no dejar espacios entre ellos.
- ✓ El nombre de la variable no puede coincidir con palabras clave pertenecientes a la propia sintaxis de JavaScript, ni con las palabras reservadas.
- ✓ Se puede utilizar la palabra clave **var** seguida del nombre de la variable, no es necesario declarar su tipo, como en otros lenguajes.

Por ejemplo:

Variables correctas	Variables Incorrectas
var nombre;	var 1dato; //comienza con número
var Direccion;	var entrada datos; //contiene un espacio
var entrada_valor_concreto;	var while; //palabra reservada
var variable_numero_12;	Var new; //palabra reservada

Tipos de Variables

JavaScript puede manejar varios tipos de datos distintos decidiendo por nosotros el tipo de variable que deberá emplear durante la ejecución del script. Los 4 tipos de variables son:

- ✓ Variables de cadena (aquellas que contiene texto y se delimitan mediante comillas)
- ✓ Variables numéricas (aquellas que contienen números enteros o de coma flotantes)
- ✓ Variables booleanas (se especifican mediante los valores true o false)
- ✓ Variables undefined (variables que han sido creadas, pero no tienen ningún valor asignado)

Y el quinto tipo podría ser los datos NULOS. Los valores (null) se utilizan para comprobar si una variable se le ha asignado un valor o no. Así pues, null representa un valor nulo para cualquier tipo de variable.

Entonces la sintaxis de JavaScript es el conjunto de reglas, cómo se construyen los programas de JavaScript:

```
var a, b, c; //Así se declaran las variables
a=1; y=2; //Así se asignan valores
c=a+b; //Así se calculan valores
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



La sintaxis de JavaScript define dos tipos de valores: **fijos y variables**. Los valores fijos son llamados literales y los variables son llamados variables.

Los **operadores aritméticos** que usa JavaScript para calcular valores son: +, -, *, /. El operador de asignación (=) ya fue mostrado arriba.

En JavaScript una expresión es una combinación de valores, variables y operadores, con las cuales calculas un valor. Por ejemplo: $3*2$ ó **var a; a=2; var resultado = a*2;**

En lo que se refiere a las Estructuras de Control, se manejan prácticamente igual que otros lenguajes de programación, por lo que ustedes deberán revisarlos con los ejemplos que están en la página de w3schools.

Es importante mencionar que JavaScript trabaja con **objetos**, los cuales son escritos entre {}, sus propiedades son escritas en pares **nombre:valor**, separadas por comas (,).

```
Var alumno = {apePat: "Escamilla", apeMat: "Miranda", nombre: "Patricia",  
              numBoleta:2022630103};
```

2. Modelo de Eventos

En la programación tradicional, las aplicaciones se ejecutan secuencialmente de principio a fin para producir sus resultados. Sin embargo, en la actualidad el modelo predominante es el de la programación basada en eventos. Los scripts y programas esperan sin realizar ninguna tarea hasta que se produzca un evento. Una vez producido, ejecutan alguna tarea asociada a la aparición de ese evento y cuando concluye, el script o programa vuelve al estado de espera.

Es decir, **se llama evento al suceso que ocurre en el sistema cuando un usuario ejecuta algún tipo de acción**. Un evento debe asociarse a un elemento HTML, no a un código JavaScript.

El modelo simple de eventos se introdujo en la versión 4 del estándar HTML y se considera parte del nivel más básico de DOM. Aunque sus características son limitadas, es el único modelo que es compatible con todos los navegadores y, por tanto, el único que permite crear aplicaciones que funcionan de la misma manera en todos los navegadores.

Tipos de eventos

Cada elemento HTML tiene definida su propia lista de posibles eventos que se le pueden asignar. Un mismo tipo de evento (por ejemplo, pinchar el botón izquierdo del ratón) puede estar definido para varios elementos HTML y un mismo elemento HTML puede tener asociados diferentes eventos.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



El nombre de los eventos se construye mediante el prefijo **on**, seguido del nombre en inglés de la acción asociada al evento. Así, el evento de pinchar un elemento con el ratón se denomina **onclick** y el evento asociado a la acción de mover el ratón se denomina **onmousemove**.

La siguiente tabla resume los eventos más importantes definidos por JavaScript:

Evento	Descripción	Elementos para los que está definido
<code>onblur</code>	Un elemento pierde el foco	<code><button></code> , <code><input></code> , <code><label></code> , <code><select></code> , <code><textarea></code> , <code><body></code>
<code>onchange</code>	Un elemento ha sido modificado	<code><input></code> , <code><select></code> , <code><textarea></code>
<code>onclick</code>	Pulsar y soltar el ratón	Todos los elementos
<code>ondblclick</code>	Pulsar dos veces seguidas con el ratón	Todos los elementos
<code>onfocus</code>	Un elemento obtiene el foco	<code><button></code> , <code><input></code> , <code><label></code> , <code><select></code> , <code><textarea></code> , <code><body></code>
<code>onkeydown</code>	Pulsar una tecla y no soltarla	Elementos de formulario y <code><body></code>
<code>onkeypress</code>	Pulsar una tecla	Elementos de formulario y <code><body></code>
<code>onkeyup</code>	Soltar una tecla pulsada	Elementos de formulario y <code><body></code>
<code>onload</code>	Página cargada completamente	<code><body></code>
<code>onmousedown</code>	Pulsar un botón del ratón y no soltarlo	Todos los elementos
<code>onmousemove</code>	Mover el ratón	Todos los elementos
<code>onmouseout</code>	El ratón "sale" del elemento	Todos los elementos
<code>onmouseover</code>	El ratón "entra" en el elemento	Todos los elementos
<code>onmouseup</code>	Soltar el botón del ratón	Todos los elementos
<code>onreset</code>	Inicializar el formulario	<code><form></code>
<code>onresize</code>	Modificar el tamaño de la ventana	<code><body></code>
<code>onselect</code>	Seleccionar un texto	<code><input></code> , <code><textarea></code>
<code>onsubmit</code>	Enviar el formulario	<code><form></code>
<code>onunload</code>	Se abandona la página, por ejemplo al cerrar el navegador	<code><body></code>

Los eventos más utilizados en las aplicaciones web tradicionales son **onload** para esperar a que se cargue la página por completo, los eventos **onclick**, **onmouseover**, **onmouseout** para controlar el ratón y **onsubmit** para controlar el envío de los formularios.

Algunos eventos de la tabla anterior (**onclick**, **onkeydown**, **onkeypress**, **onreset**, **onsubmit**) permiten evitar la "acción por defecto" de ese evento.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Las acciones típicas que realiza un usuario en una página web pueden dar lugar a una sucesión de eventos. Al pulsar por ejemplo sobre un botón de tipo `<input type= "submit">` se desencadenan los eventos **onmousedown**, **onclick**, **onmouseup** y **onsubmit** de forma consecutiva.

Un evento de JavaScript por sí mismo carece de utilidad. Para que los eventos resulten útiles, se deben asociar funciones o código JavaScript a cada evento. De esta forma, cuando se produce un evento se ejecuta el código indicado, por lo que la aplicación puede responder ante cualquier evento que se produzca durante su ejecución.

Las funciones o código JavaScript que se definen para cada evento se denominan *manejador de eventos* (*event handlers* en inglés) y como JavaScript es un lenguaje muy flexible, existen varias formas diferentes de indicar los manejadores:

- ✓ Manejadores como atributos de los elementos HTML.
- ✓ Manejadores como funciones JavaScript externas.

Les comparto un ejemplo de cómo utilizar los eventos `onmouseover` y `onmouseout`, los cuales fueron explicados brevemente al inicio de esta página.

Su sintaxis es la siguiente:

``

Dichos eventos actúan, generando el típico efecto de cambiar la imagen cuando el cursor pasa sobre ella:



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



```
<> pruebaEventos.html X
<> pruebaEventos.html > html > body > p > img#icono
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>::: Prueba de eventos :::</title>
7
8      <script type="text/javascript">
9          imagen1 = new Image;
10         imagen1.src = "js.png";
11
12         imagen2 = new Image;
13         imagen2.src = "jQuery.png";
14
15         let i=1;
16         function cambiaImagen(){
17             if(i==1){
18                 document.images['icono'].src = imagen2.src;
19                 i=2;
20             }
21             else{
22                 document.images['icono'].src = imagen1.src;
23                 i=1;
24             }
25         }
26     </script>
27
28 </head>
29 <body>
30     <h2>Eventos onmouseover y onmouseout</h2>
31     <p></p>
33 </body>
34 </html>
```

3. Creación de Clases con JavaScript

JavaScript se convirtió en un estándar ECMA en 1997. ECMA-262 es el nombre oficial del estándar. ECMAScript (es el **estándar** que a partir del año 2015 a la actualidad se encarga de **regir como debe ser interpretado y funcionar el lenguaje JavaScript**), es el nombre oficial del idioma.

ECMAScript2015, también conocido como ES6, introdujo clases.

Una clase es un tipo de función, pero en lugar de usar *function* de palabra clave para iniciarla, usamos la palabra clave **class** y las propiedades se asignan dentro de un método **constructor()**.

Definición de clases

Use la palabra clave **class** para crear una clase y siempre agregue el método **constructor()**.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Se llama método constructor cada vez que se inicializa el objeto de clase.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class</h2>

<p>In this example we demonstrate a simple class definition and how to use it.</p>

<p id="demo"></p>

<script>
class Car {
  constructor(brand) {
    this.carname = brand;
  }
}

mycar = new Car("Ford");

document.getElementById("demo").innerHTML = mycar.carname;
</script>

</body>
</html>
```

Métodos

El método constructor es especial, es donde se inicializan las propiedades, se llama automáticamente cuando se inicia una clase y debe tener el nombre exacto “constructor”, de hecho, si no tiene un método constructor, JavaScript agregará un método de constructor invisible y vacío.

También es libre de hacer sus propios métodos, la sintaxis debe ser familiar.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Class Method</h2>

<p>Demonstration of how to add a method in a JavaScript Class.</p>

<p id="demo"></p>

<script>
class Car {
  constructor(brand) {
    this.carname = brand;
  }
  present() {
    return "I have a " + this.carname;
  }
}

mycar = new Car("Ford");

document.getElementById("demo").innerHTML = mycar.present();
</script>

</body>
</html>
```

JavaScript Class Method

Demonstration of how to add a method in a JavaScript Class.

I have a Ford

También tiene métodos estáticos, herencia, reglas de strict mode, etc., pero esto lo podrán revisar en la página de [w3schools](https://www.w3schools.com), para que practiquen con los ejemplos del código proporcionados.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



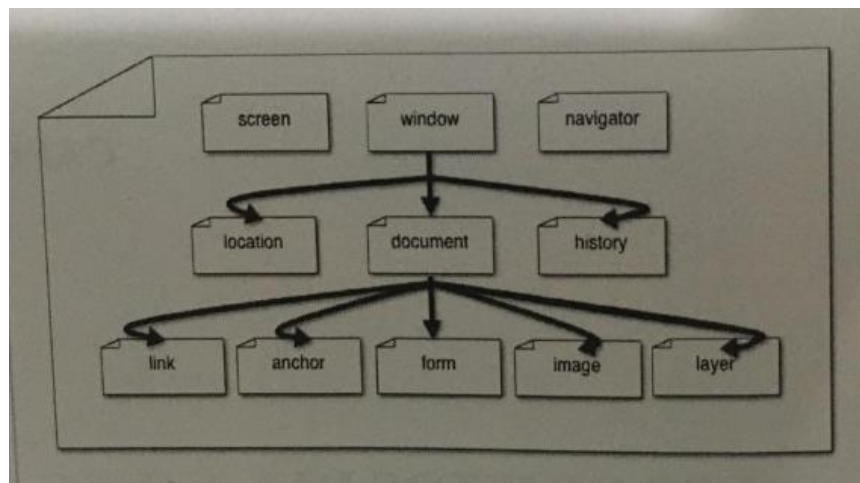
4. Modelo de objetos del Navegador

Cuando se carga una página en un navegador se crean un número de objetos característicos del navegador según el contenido de dicha página.

A continuación, les enlisto los objetos y propiedades que tiene un documento:

- ✓ **Window.** Es el objeto de más alto nivel, contiene las propiedades de la ventana y en el supuesto de trabajar con frames, un objeto window es generado para cada frame.
- ✓ **Navigator.** Contiene las propiedades del navegador.
- ✓ **Location.** Contiene las propiedades de la URL activa.
- ✓ **History.** Objeto que contiene las propiedades que representan a las URL que el usuario ha visitado anteriormente. Es una especie de *caché*.
- ✓ **Document.** Este objeto contiene todas las propiedades del documento actual, como pueden ser su color de fondo, enlaces, imágenes, etc.
- ✓ **Screen.** Contiene información referente a la resolución de la pantalla que muestra la URL.

Los objetos en el navegador se rigen por una jerarquía que refleja la estructura de los documentos XHTML. Es decir, el objeto window que es el de más alto nivel tendría a un objeto location, history y document como descendientes.



Para entenderlo mejor; imaginemos un documento XHTML (object document) que contiene un formulario llamado *miformulario*. Ahora bien, para hacer referencia al formulario se deberá escribir:

document.miformulario



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



El OBJETO window

El objeto window posee una serie de propiedades que determinan características básicas de la ventana y sus componentes. A continuación, aparecen reseñadas las propiedades más elementales:

- ✓ **closed.** Propiedad que determina si una ventana se ha cerrado.
- ✓ **defaultStatus.** Propiedad que contiene el mensaje estándar que aparece en la barra de estado de Windows.
- ✓ **frames.** Es una matriz que representa todos los frames de la ventana.
- ✓ **innerHeight.** Altura del área del documento interno de la ventana.
- ✓ **innerWidth.** Anchura del área del documento interno de la ventana.
- ✓ **length.** Esta propiedad contiene el número de frames de la ventana.
- ✓ **name.** Contiene el nombre de la ventana.
- ✓ **outerHeight.** Altura de la totalidad de la ventana.
- ✓ **outerWidth.** La anchura de la totalidad de la ventana.
- ✓ **parent.** Hace referencia a la ventana con un código <frameset>
- ✓ **self.** Propiedad que hace referencia a la ventana activa.
- ✓ **top.** Hace referencia a la ventana superior del navegador.
- ✓ **status.** Determina el mensaje que aparece en la barra de estado del navegador.
- ✓ **window.** Al igual que self, hace referencia a la ventana activa.

El objeto window también posee una serie de métodos que permiten ejecutar funciones específicas con las ventanas, como por ejemplo crear ventanas y cuadros de diálogo.

Algunos de ellos son:

- ✓ **open()** y **close()**. Métodos que abren y cierran una ventana.
- ✓ **back()**. Retrocede a la página anterior.
- ✓ **blur()**. Quita el foco de la ventana especificada.
- ✓ **captureEvents()**. Captura todos los eventos de un determinado tipo.
- ✓ **clearInterval()**. Cancela el tiempo de espera establecido mediante el método setInterval().
- ✓ **alert()**. Método que muestra una ventana de diálogo con un mensaje y el botón aceptar.
- ✓ **confirm()**. Método similar al anterior, pero mostrando dos botones: Aceptar y Cancelar.
- ✓ **find()**. Abre una ventana de diálogo que permite efectuar búsquedas.
- ✓ **prompt()**. Método que representa una ventana de diálogo con un mensaje y un campo de entrada.
- ✓ **setTimeout()**. Este método retrasa la ejecución de una instrucción.
- ✓ **clearTimeout()**. Permite anular el timeout fijado con el método anterior.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



EL OBJETO **location**

El objeto **location** contiene toda la información sobre el URL que se está visualizando, así como todos los detalles de esa dirección (puerto, protocolo, etc.).

Su sintaxis: ***window.location.propiedad***

Las propiedades más importantes del objeto **location** son:

- ✓ **href.** Hace referencia a la dirección completa, bien sea para visualizar la URL actual o para encaminarse a una nueva:
document.write(location.href);
location.href= "<http://www.google.com>";
- ✓ **protocol.** Especifica el inicio de la dirección: http:, file: y ftp:.
document.write(location.protocol);
- ✓ **host.** Esta propiedad muestra el nombre del servidor donde está alojada la URL y su puerto.
document.write(location.host);
- ✓ **hostname.** Igual que la anterior pero sin mostrar el puerto.
document.write(location.hostname);
- ✓ **hash.** Especifica el nombre del enlace en la URL si existe.
document.write(location.hash);
- ✓ **port.** Indica el puerto de la URL, en caso de existir.
document.write(location.port);
- ✓ **search.** Devuelve la sección de la URL que va después del signo de interrogación "?" si existe. De la dirección <http://search.yahoo.com/bin/search?p=coches> obtendríamos p=coches.
location.search;

Este objeto además contiene dos métodos:

- ✓ **reload().** Permite recargar la página actual.
location.reload();
- ✓ **replace().** Permite cambiar la URL actual por la especificada en el argumento.
location.replace("http://www.ra-ma.com");

EL OBJETO **screen**

El objeto **screen** permite obtener del navegador que está visualizando una URL, información acerca de la resolución de la pantalla. Su sintaxis es la siguiente:

screen.propiedad



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Sus propiedades son:

- ✓ **height.** Devuelve la altura de la pantalla en píxeles.
- ✓ **width.** Devuelve el ancho de la pantalla en píxeles.
- ✓ **colorDepth.** Devuelve la profundidad de bits de la paleta de colores. Contiene el número de bits de color por píxel. 2 significa 16 colores, 8 significa 256 colores y 32 es el color real
- ✓ **pixelDepth.** Devuelve la resolución de color (en bits por píxel) de la pantalla.
- ✓ **availHeight.** Especifica la altura útil de la pantalla en píxeles.
- ✓ **availWidth.** Especifica la anchura útil de la pantalla en píxeles.

El OBJETO document

El objeto **document** hace referencia a todo el contenido de un documento HTML.

Todas las propiedades de este objeto hacen referencia a determinadas características de la página, como su color de fondo, su título, etc. Algunas de las más utilizadas son:

- ✓ **bgColor.** Color de fondo.
- ✓ **fgColor.** Color de texto.
- ✓ **vlinkColor.** Color de los enlaces visitados.
- ✓ **alinkColor.** Color del enlace en el momento de la selección.
- ✓ **cookie.** Usado para identificar el valor de una cookie.
- ✓ **domain.** Indica el nombre del dominio del servidor de documentos.
- ✓ **embeds.** Matriz que contiene todos los plugins de un documento
- ✓ **fileCreatedDate.** Indica cuando fue creado el documento HTML.
- ✓ **referrer.** Especifica la dirección URL desde la que el usuario a derivado a la actual, normalmente a través de un enlace.
- ✓ **fileModifiedDate.** Use este valor para mostrar la fecha de los últimos cambios.
- ✓ **title.** Indica el nombre del documento actual definido en las etiquetas title.
- ✓ **URL.** La dirección del documento actual.

También podemos trabajar con algunos de sus métodos para controlar el proceso de abrir y cerrar un documento.

- ✓ **open().** Abre un nuevo documento.
- ✓ **close().** Cierra el documento.
- ✓ **write().** Permite escribir en un documento.
- ✓ **writeln().** Permite escribir en un documento insertando un carácter de línea nueva al final.

```
document.open(""); //Abre una ventana en blanco  
document.write("<b>Escribe este contenido en negritas</b>")  
document.close(); //Cerrar ventana
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



EL OBJETO history

Este objeto contiene información sobre los enlaces que el usuario ha visitado. Su utilidad más evidente es la de generar botones de avance y retroceso. Su sintaxis es la siguiente:

history.propiedad

La propiedad `length` contiene el número de elementos que contiene la lista de URL visitadas.

document.write(history.length);

Los métodos son los siguientes:

- ✓ **back()**. Carga la URL anterior a la actual.
- ✓ **forward()**. Carga la URL siguiente de la lista.
- ✓ **go()**. Muestra una URL de la lista `history` según un valor índice.

history.go(-2) //Retrocede 2 ítems en la lista

history.go(2) //Avanza 2 ítems en la lista

`<a href="javascript:history.go(-3)">Retrocede 3 páginas</a>`

`<a href="javascript:history.go('google.es')">Acceder a Google.es</a>`

EL OBJETO navigator

Este objeto permite obtener datos del navegador que está visualizando un documento.

Mediante sus propiedades podremos saber el tipo de navegador, su versión, bajo qué sistema operativo trabaja, etc. Su sintaxis es:

navigator.propiedad

Sus propiedades son:

- ✓ **appName**. Devuelve el nombre codificado dado el navegador.
- ✓ **appName**. Nombre del navegador.
- ✓ **appVersion**. Facilita la versión del navegador.
- ✓ **browserLanguage**. Idioma del navegador.
- ✓ **cookieEnabled**. Informa si el navegador tiene activadas las cookies.
- ✓ **cpuClass**. Facilita el tipo de CPU del sistema.
- ✓ **language**. Idioma del mismo.
- ✓ **platform**. Plataforma donde se está ejecutando.
- ✓ **systemLanguage**. Devuelve el idioma del sistema.
- ✓ **userAgent**. Encabezado enviado como parte del protocolo HTTP.
- ✓ **userLanguage**. Devuelve el idioma actual.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Las propiedades del objeto navigator son de lectura, es decir, no podrán modificar o asignar valor a ninguna de ellas.

Este objeto dispone de 2 métodos:

- ✓ **taintEnabled()**. Devuelve *false* y se incluye en Internet Explorer por compatibilidad con Netscape.
- ✓ **javaEnabled()**. Devuelve *true* si el intérprete de Java está activado.

5. Manejo de Expresiones Regulares

Las expresiones regulares son patrones que permiten buscar coincidencias con combinaciones de caracteres dentro de cadenas de texto. La filosofía de las expresiones regulares es trabajar con patrones para buscar, copiar o reemplazar fragmentos de texto.

En JavaScript, las expresiones regulares son también objetos.

Y mediante las expresiones regulares podemos:

- ✓ Comprobar el formato de una cadena, por ejemplo, números de teléfono, tarjetas de crédito, fechas, etc.
- ✓ Reemplazar o extraer fragmentos de texto.

Para que se comprenda mejor, piense en su disco duro y el buscador de archivos del sistema. Si buscamos algo similar a *lista**, obtendría, por ejemplo, *lista1*, *lista2*, *listados*, etc. Pues bien, la filosofía de las expresiones regulares es la misma y son soportadas por prácticamente todos los navegadores.

Creación de Patrones

Las expresiones regulares pueden construirse de dos maneras distintas. La primera de ellas es mediante una expresión literal:

expresión=/patrón/

La segunda es creando un objeto:

Los patrones se pueden utilizar con los métodos *exec()* y *test()* del objeto **RegExp**, o bien con los métodos *match()*, *replace()*, *search()* y *split()* del objeto **String**.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Por ejemplo:

```
txt="La casa bonita del campo era grande y tenía verdes árboles";  
document.write(txt.match(/campo/));
```

El script devolverá el valor *campo*, ya que dicha cadena se encuentra en la variable txt. Si el patrón no se encuentra, se devuelve el valor null. El patrón a buscar se ha colocado entre los signos / /.

También podemos utilizar los siguientes caracteres especiales que nos van a permitir generar patrones de búsqueda.

- \ Permite buscar retornos de carro, tabuladores, etc., es decir, caracteres de cadena.
- ^ Permite localizar el comienzo de una cadena. `Document.write(txt.match(/^Ma/));`
- \$ Se utiliza para localizar el final de una línea – `document.write(txt.match(/es$/));`
- * Encuentra las coincidencias de los caracteres que le preceden o o más veces.
- + Encuentra una o más coincidencias de los caracteres que le preceden.
- ? Encuentra o o una coincidencia de los caracteres que le preceden,
- . Localiza cualquier carácter siempre que no sea comienzo de línea.
- (a) Localiza coincidencias con el patrón *a* recordando el mismo para usarlo posteriormente.
- a|b Localiza cualquiera de los dos patrones (*a* o *b*).
- {n} Encuentra *n* coincidencias de los caracteres que le preceden.
- {n,} Encuentra *n* coincidencias como mínimo de los caracteres precedentes.
- {n,m} Permite localizar un mínimo (*n*) y un máximo (*m*) de coincidencias de los caracteres precedentes.
- [**caracteres**] Localiza cualquiera de los caracteres indicados entre los corchetes. Se puede especificar un rango de caracteres con el guión (-).
- [**^caracteres**] Indica los caracteres que no deben de aparecer.
- [**\b**] Localiza coincidencias con el carácter de retroceso.
- \b Localiza cualquier límite de palabra.
- \w Encuentra cualquier carácter subrayado.
- \W Encuentra caracteres que no sean de palabra.
- \s Localiza cualquier espacio en blanco.
- \S Localiza cualquier carácter especial que no sea blanco.
- \v Localiza una tabulación vertical.
- \d Localiza dígitos en una cadena.
- \D Localiza cualquier carácter que no sea un dígito.
- \r Localiza un retorno de carro.
- \\ Localiza la barra inclinada.

La verdadera potencia de las expresiones regulares se obtiene realizando patrones mediante la combinación de éstos.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



Por ejemplo:

c.l	Una "c" seguida de un carácter cualquiera y una "l", "cel", "cal",
^c	Una "c" al principio de la cadena. "carlos", "celeste".
ón\$	"ón" al final de la cadena. "camión", "melón", "sifón".
^\$	Una cadena vacía.
[abc]	Una "a" o una "b" o una "c".
[^abc]	Cualquiera que no sea "a", "b" o "c".
[a-z]	Cualquier letra entre la "a" y la "z".
[^a-z]	Cualquier letra entre la "a" y la "z" que no sean letras minúsculas.
[a-z]+	Una secuencia no vacía de letras minúsculas.
p.*ta	Coincidencias con "pta", "peta", "pita", etc.
p.+ta	Coincidencias como la anterior a excepción de "pta".
^[\t]*\$	Una línea en blanco, o combinaciones de espacios y tabuladores.
[+]?ld*.?ld*	Números decimales.
(\d{1,2})V(\d{1,2})V(\d{2,4})	Formato fecha o número tipo 28/04/200, 01/01/01 ó 11/11/1111.
Sonia Lydia	Coincidencias con "Sonia" o "Lydia".
(zapa pimien)tos	Coincidencias con "zapatos" o "pimientos".
[01]	Un "0" o un "1".
pata peta pita	Coincidencias con "pata", "peta" o "pita".
d(e a i)do	Coincidencias con "ddo", "dado" o "dido".

Ahora el siguiente script, se encarga de localizar en el campo *correo* el carácter arroba:

EjemExpReg.html X

EjemExpReg.html > html > body > script > arroba

```
1 <html lang="es">
2 <head>
3 </head>
4 <body>
5 <h3>Verificando el correo</h3>
6 <form id="datos">
7   <label for="correo">Correo:</label>
8   <input type="text" name="correo" id="correo"
9     placeholder="Ingresa tu correo"><br>
10   <label for="resultado">Muestro el arroba:</label>
11   <input type="text" id="resultado"><br>
12   <input type="button" value="Verificar" onclick="arroba();"
13 </form>
14
15 <script type="text/javascript">
16   function arroba(){
17     var txt = document.forms.datos.correo.value;
18     var resultado = txt.match(/@/);
19     if(resultado == '@'){
20       document.forms.datos.resultado.value = "Tu correo
21       es correcto";
22     }else{
23       document.forms.datos.resultado.value = "Tu correo
24       es incorrecto";
25     }
26   }
27 </script>
28 </body>
```

Ej. Expresión Regular Match

127.0.0.1:5500/ejem...

Verificando el correo

Correo: pescamilla@ipn.mx

Muestro el arroba: Tu correo es correcto

Verificar

Ej. Expresión Regular Match

127.0.0.1:5500/ejem...

Verificando el correo

Correo: pescamillaipn.mx

Muestro el arroba: Tu correo es incorrecto

Verificar

Sabemos que el campo de tipo email, ya nos verifica esto, sin embargo, solo es un ejemplo de cómo podemos utilizar las expresiones regulares, por lo que les corresponde a ustedes hacer más ejercicios.

A continuación, podrán observar que existe un objeto (el **objeto RegExp**) que se utiliza para comprobar si las cadenas se ajustan a ciertos patrones formalizados como expresiones regulares.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web Unidad Temática V. JavaScript



JavaScript usa este objeto para trabajar con patrones de expresiones regulares. Éstos se crean como cualquier otro objeto.

```
var patron = [abc]/i  
var patron2 = new RegExp("[abc]", "i");
```

Observen cómo ahora los patrones llevan un añadido (flag) para ajustar la forma de buscar.

Veán a continuación los flags disponibles y su significado:

- g** Explora la cadena completa.
- i** Cualquier carácter sin precisar mayúsculas o minúsculas.
- m** Permite usar varios **^** y **\$** en la definición del patrón.
- s** Incluye un salto de línea en el carácter especial punto (**.**).
- x** Ignora los espacios en el patrón.

El objeto **RegExp** tiene dos métodos con los que se pueden utilizar los patrones, como se comentó al inicio de este tema, los cuales estudiaremos a continuación:

El Método **exec()**

Ejecuta una búsqueda en una cadena mediante un patrón de expresión regular devolviendo una matriz (arreglo) con los resultados de la búsqueda.

RegExp.exec(cadena)

```
expr=/r.*ta/g;  
matriz=expr.exec("La canción dice rata reta rita rota ruta");  
document.write(matriz);
```

Devolvería: rata reta rita rota ruta

El siguiente ejemplo funcionaría exactamente igual:

```
expr=new RegExp("r.*ta","g");  
matriz=expr.exec("La canción dice rata reta rita rota ruta");  
document.write(matriz);
```

Si el método no encuentra una coincidencia, devuelve el valor *null*.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript

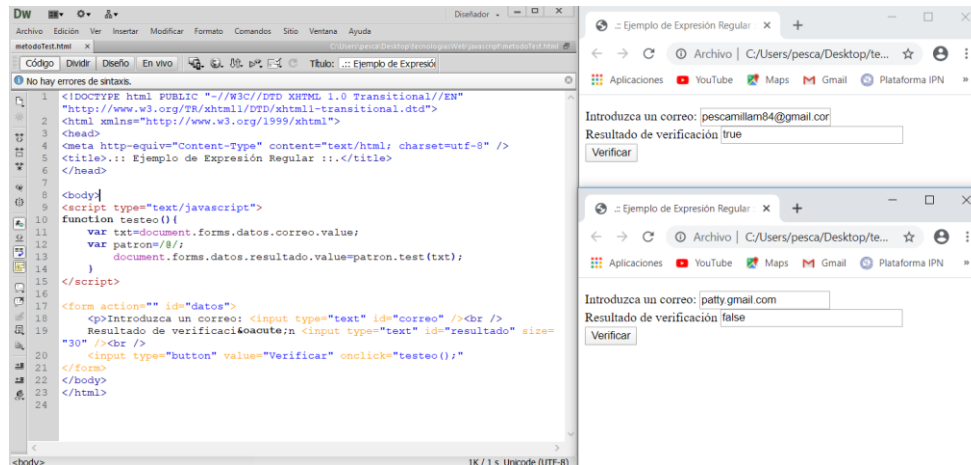


El Método test()

Este método devuelve un valor *true* si se ha encontrado alguna coincidencia en la cadena o *false* en caso contrario.

objeto.test(cadena)

El siguiente script devolverá *true* si localiza en la cadena *txt* una arroba y *false* si no encuentra ninguna.



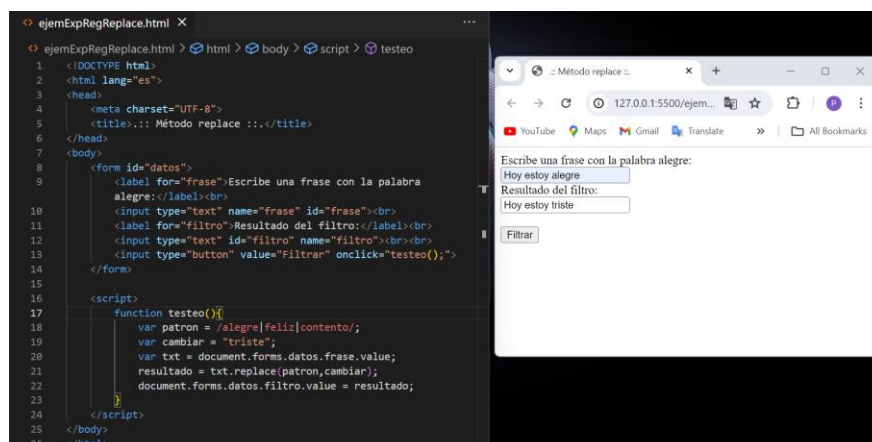
El Método replace()

Reemplaza el texto encontrado en la búsqueda de una expresión normal.

objeto.replace(cadena1,cadena2)

Este método sustituye las coincidencias del argumento *cadena1* por las especificadas en el argumento *cadena2*. Si no existen coincidencias, se devuelve el valor del argumento *cadena1*.

Por ejemplo:





U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript

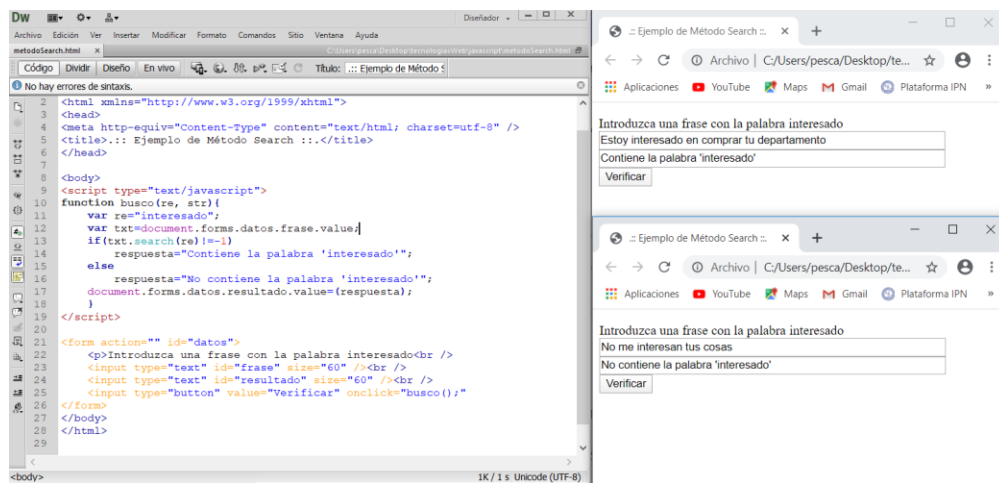


El Método `search()`

Este método busca coincidencias en una cadena.

objeto.search(cadena)

El siguiente script busca en la cadena *txt* coincidencias con la palabra *interesado*. Si ésta existe, el método devuelve un valor *true* que es pasado a una sentencia *if* para mostrar la respuesta adecuada. Si no se encuentra ninguna coincidencia, el método devuelve un valor *false*.



El Método `split()`

Este método devuelve una matriz con los elementos en que queda dividida la cadena según el separador indicado en el argumento *n*.

objeto.split(n)

Es un método propio del *objeto String()*, pero se presenta aquí a modo de ejemplo para comprobar cómo pueden aplicarse las expresiones regulares para localizar caracteres.

En el siguiente ejemplo, el separador de los elementos de la cadena se ha definido como espacio en blanco `/\s/`.

```
ciudades = "Barcelona Tarragona Lleida Girona";  
document.write(ciudades+"<br />");  
re=/\s/;  
resultado=ciudades.split(re);  
document.write(resultado);
```

El resultado es Barcelona, Tarragona, Lleida, Girona.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática V. JavaScript



De igual forma, podrán consultar y probar los ejercicios de [expresiones regulares](#), en donde verán código más familiar, por citar un ejemplo, se muestra el uso del método **replace()**, de la siguiente forma:

```
<!DOCTYPE html>
<html>

<body>

<h2>JavaScript String Methods</h2>
<p>Replace "Microsoft" with "W3Schools" in the paragraph below:
</p>

<button onclick="myFunction()">Try it</button>

<p id="demo">Please visit Microsoft!</p>

<script>
function myFunction() {
  var str = document.getElementById("demo").innerHTML;
  var txt = str.replace("Microsoft","W3Schools");
  document.getElementById("demo").innerHTML = txt;
}
</script>

</body>
</html>
```

JavaScript String Methods

Replace "Microsoft" with "W3Schools" in the paragraph below:

Please visit Microsoft!

JavaScript String Methods

Replace "Microsoft" with "W3Schools" in the paragraph below:

Please visit W3Schools!

El script anterior reemplaza la palabra “Microsoft” por “W3Schools” en la frase: Please visit Microsoft!, al dar clic en el botón Try it y accedemos a ese párrafo a través del método getElementById de DOM. Así como este método, podrán revisar los otros y los objetos del método RegExp mostrados en este documento.

Recuerden que, en la Unidad Temática anterior, nosotros ya revisamos el Modelo de Objetos de Documento (DOM) de W3C, que es una plataforma y una interfaz de lenguaje neutral que permite que los programas y los scripts accedan y actualicen dinámicamente el contenido, la estructura y estilo de un documento. Sin embargo, si creen necesario repasar el tema aquí les dejo la liga: [JavaScript - DOM](#)